
No. 1
OCT.
I LOVE SQL
0110110
SELECT * FROM love;
SELECT * FROM love;
{...}
WHERE name='LOVE';
SELECT * FROM love
{...}

SQL і NoSQL: теорія для співбесіди

Рівень мідла, без фанатизму. Теорія коротко, потім практика в репо sql-drills.

Як користуватись. Кожен розділ закінчується блоком «Практика»: там написано, які вправи з репо відповідають цій темі. Читаєш розділ, потім йдеш у репо й пишеш запити.

```
npm install
npm run list            # усі вправи і що пройдено
npm run task 05         # завдання і очікуваний результат
# пишеш запит у drills/05-inner-join/my.sql
npm run check 05        # перевірка
npm run hint 05         # підказки
```

Усі приклади нижче крутяться на тих самих таблицях, що й вправи (`users`, `orders`, `big_orders`). Виводи запитів справжні, прогнані на seed з репо. Команда `npm run schema` показує всі таблиці.

Розділ	Тема	Вправи в репо
1	Реляційна модель, ключі, нормалізація	дивись схему: npm run schema
2	Порядок виконання запиту, group by, having	01-04
3	JOIN і пастки з NULL	05-07
4	UNION і UNION ALL	08-10
5	CASE, EXISTS, CTE, віконні функції	11-17, додатково 19
6	Індекси і EXPLAIN	21, додатково 22
7	Транзакції, ACID, рівні ізоляції	в репо нема, питають усно
8	NoSQL: які бувають і для чого	в репо нема, питають усно
9	SQL чи NoSQL: як вибирати	в репо нема, питають усно
	Фінальна перевірка всього SQL	20 (бос-вправа)

Частина 1. SQL

1. Реляційна модель

Дані лежать у таблицях. Рядок це один запис, колонка це одна властивість. Таблиці зв'язують ключами:

- **Primary key (PK)**: унікально ідентифікує рядок. У `users` це `id`.
- **Foreign key (FK)**: посилання на PK іншої таблиці. `orders.user_id` вказує на `users.id`, тож неможливо створити замовлення неіснуючого юзера.
- **Зв'язки**: один-до-багатьох (юзер і його замовлення), багато-до-багатьох (потрібна проміжна таблиця з двома FK).

Нормалізація це прибирання дублювання, щоб один факт лежав в одному місці:

- **1NF**: у клітинці одне значення, не список («phones: 1,2,3» так не можна).
- **2NF**: усе, що не ключ, залежить від усього складеного ключа, а не від його частини.
- **3NF**: неключові колонки залежать від ключа, а не одна від одної (країна юзера лежить в users, а не копіюється в кожне замовлення).

Денормалізація це свідоме дублювання заради швидкості читання. Ціна: треба тримати копії синхронними.

> **Практика в репо (sql-drills)**
> Подивись схему: `npm run schema`. Знайди PK і FK між `users` і `orders` у `seed/seed.sql`.

2. Порядок виконання запиту

Пишемо запит у одному порядку, а база виконує в іншому. Це найпопулярніше питання про SQL.

```
пишемо:     SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT
виконується: FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

- **WHERE** фільтрує рядки до групування, **HAVING** фільтрує групи після нього.
- Через це в WHERE не можна використати аліас з SELECT (SELECT ще не виконався), а в ORDER BY можна.

Приклад: користувачі, чия сума оплачених замовлень більша за 100.

```
select user_id, sum(amount) as total
from orders
where status = 'paid'
group by user_id
having sum(amount) > 100
order by user_id;

user_id | total
--------+------
1       | 170
2       | 140
4       | 400
```

Спочатку WHERE лишив тільки paid, потім GROUP BY склав по юзерах, потім HAVING відкинув юзерів із сумою до 100 включно.

> **Практика в репо (sql-drills)**
> Вправи **01-04**: select + where, order by + limit, group by, having.

3. JOIN

JOIN склеює рядки двох таблиць за умовою. Різниця між типами в тому, що відбувається з рядками без пари.

Тип	Що повертає	Коли
INNER JOIN	тільки рядки, що мають пару в обох таблицях	юзери з замовленнями
LEFT JOIN	усі рядки лівої, а де пари нема, праві колонки NULL	усі юзери і їхні замовлення, якщо є
FULL OUTER JOIN	усі рядки обох таблиць, пари нема з будь-якого боку дає NULL	звірка двох списків

Anti-join: хто не має замовлень. LEFT JOIN і фільтр на NULL з правого боку:

```
select u.email
from users u
left join orders o on o.user_id = u.id
where o.id is null
order by u.id;

email
----------
cid@x.com
fira@x.com
```

Пастка з NULL. NULL означає «невідомо», тому будь-яке порівняння з ним дає не true, а невідомо. Через це `NOT IN` із підзапитом, що повертає хоч один NULL, не поверне нічого:

```
select count(*) as n from users
where id not in (select null::int);

n
-
0

-- хоча в таблиці 6 юзерів. Безпечніше: NOT EXISTS або LEFT JOIN ... IS NULL.
```

> **Практика в репо (sql-drills)**
> Вправи **05** (inner join), **06** (left join як anti-join), **07** (full outer join).

4. UNION і UNION ALL

UNION склеює результати двох запитів **по вертикалі** (JOIN склеює по горизонталі). Колонки мають збігатись за кількістю і типом.

- **UNION** прибирає дублікати (для цього треба сортувати або хешувати, тож повільніше).
- **UNION ALL** лишає все як є і швидший. Якщо дублікатів бути не може або вони потрібні, бери його.

```
A = (1, 2, 2)     B = (2, 3)

A union B      ->  1, 2, 3
A union all B  ->  1, 2, 2, 2, 3
```

Приклад з репо: платежі з трьох провайдерів (`stripe_payments`, `paypal_payments`, `liqpay_payments`) збирають в одну таблицю через UNION ALL.

> **Практика в репо (sql-drills)**
> Вправи **08** (union), **09** (union чи union all), **10** (три провайдери в одну таблицю).

5. CASE, EXISTS, CTE, віконні функції

- **CASE WHEN**: умова всередині запиту (аналог if). **COALESCE(a, b)**: перше не-NULL значення, зручно для значень за замовчуванням.
- **EXISTS**: чи є хоч один рядок у підзапиті. Зупиняється на першому знайденому, тому часто швидший за IN і безпечний з NULL.
- **CTE (WITH)**: дає проміжному результату ім'я, запит читається кроками зверху вниз. Рекурсивний CTE обходить дерева (категорії, оргструктура).

Віконні функції рахують по групі рядків, але **не схлопують** їх (на відміну від GROUP BY). Рядки лишаються, до кожного додається результат.

```
select user_id, amount,
       row_number() over (partition by user_id order by amount desc) as rn
from orders where status = 'paid';

user_id | amount | rn
--------+--------+---
1       | 100    | 1
1       | 50     | 2
1       | 20     | 3
2       | 70     | 1
2       | 70     | 2

-- partition by ділить на групи (по юзеру), order by нумерує всередині групи
```

- **row_number**: 1,2,3,4 без повторів. **rank**: при рівних значеннях однаковий номер, далі пропуск (1,1,3). **dense_rank**: однаковий номер без пропуску (1,1,2).
- **lag / lead**: значення з попереднього / наступного рядка.
- Типовий use case: «топ-N у кожній групі» (filter по `rn <= N` у зовнішньому запиті).

> **Практика в репо (sql-drills)**
> Вправи **11** (case, coalesce), **12** (exists), **13** (CTE), **14-16** (row_number, rank/dense_rank, нарастаюча сума), **17** (distinct on, це тільки Postgres), додатково **19** (recursive CTE).
> Після всього: **20**, бос-вправа, де join, group by, CTE і row_number в одному запиті.

6. Індекси і EXPLAIN

Індекс це окрема структура, що дозволяє знайти рядки без читання всієї таблиці. Найчастіший тип, **B-tree**, добре працює для =, діапазонів (>, BETWEEN) і сортування.

Дивимось, що робить база. У `big_orders` 100 000 рядків, шукаємо замовлення одного юзера.

```
explain select * from big_orders where user_id = 5;

Seq Scan on big_orders  (cost=0.00..1791.00 rows=100 width=16)
  Filter: (user_id = 5)
```

Seq Scan: база прочитала всю таблицю. Тепер створюємо індекс і питаємо ще раз:

```
create index idx_big_orders_user on big_orders(user_id);

explain select * from big_orders where user_id = 5;

Bitmap Heap Scan on big_orders  (cost=5.07..260.50 rows=100 width=16)
  Recheck Cond: (user_id = 5)
  ->  Bitmap Index Scan on idx_big_orders_user  (cost=0.00..5.04 rows=100 width=0)
        Index Cond: (user_id = 5)
```

Тепер індекс: орієнтовна вартість упала з 1791 до 260,5 (cost це умовні одиниці планувальника, не секунди). Для вибору 100 рядків із 100 000 Postgres обрав Bitmap Index Scan.

Коли індекс не допомагає:

- Функція над колонкою: `where user_id + 0 = 5` дає знову Seq Scan (перевірено: cost 2041), бо індекс побудований по `user_id`, а не по виразу.
- `LIKE '%abc'` (початок невідомий) B-tree не використає. `LIKE 'abc%'` може (у Postgres іноді потрібен `text_pattern_ops`).
- Таблиця мала або запит повертає більшу частину рядків: Seq Scan дешевший, планувальник це знає.
- **Складений індекс** `(a, b)` працює для фільтра по a і по `a + b`, але не по самому b (правило лівого префікса).

Ціна індексів: кожен INSERT / UPDATE / DELETE оновлює й індекси, тож їх забагато шкодить записові і займає місце. Індексують те, що часто лежить у WHERE, JOIN і ORDER BY. PK і UNIQUE мають індекс автоматично, FK у Postgres автоматично **ні**.

> **Практика в репо (sql-drills)**
> Вправа **21** (explain і індекс), додатково **22** (lateral і індекс).

7. Транзакції

Транзакція це група операцій, яка виконується як одне ціле: або всі, або жодна.

```
begin;
  update accounts set balance = balance - 100 where id = 1;
  update accounts set balance = balance + 100 where id = 2;
commit;     -- або rollback; і обидва update зникають
-- (таблиці accounts у репо нема, це умовний приклад)
```

ACID:

- **Atomicity**: усе або нічого (збій посередині відкочує всю транзакцію).
- **Consistency**: після транзакції дані задовольняють усі правила (PK, FK, CHECK).
- **Isolation**: паралельні транзакції не бачать чужих незавершених змін (рівень залежить від налаштування).
- **Durability**: після commit дані не пропадуть, навіть якщо сервер упаде (запис у журнал WAL).

Перевірка на seed: скасовуємо одне paid-замовлення в транзакції, дивимось і відкочуємо.

```
begin;
update orders set status='cancelled' where id = (select min(id) from orders where status='paid');

-- усередині транзакції:
status    | n
----------+--
cancelled | 2
paid      | 7
pending   | 1

rollback;

-- після rollback:
status    | n
----------+--
cancelled | 1
paid      | 8
pending   | 1
```

Аномалії при паралельній роботі (два клієнти, A і B):

```
Dirty read (брудне читання): B бачить зміну A, яку A потім скасував
  t=0  A: update orders set amount = 999 where id = 1   (ще без commit)
  t=1  B: select amount from orders where id = 1   -> 999
  t=2  A: rollback                                    -> B бачив неіснуюче значення

Non-repeatable read: B читає один рядок двічі і отримує різне
  t=0  B: select amount where id = 1   -> 100
  t=1  A: update amount = 200 where id = 1; commit
  t=2  B: select amount where id = 1   -> 200   (в одній транзакції B)

Phantom read: B повторює запит з умовою і з'являються нові рядки
  t=0  B: select count(*) where status = 'paid'   -> 8
  t=1  A: insert paid-замовлення; commit
  t=2  B: select count(*) where status = 'paid'   -> 9
```

Рівні ізоляції за стандартом SQL (✓ означає, що аномалія можлива):

Рівень	Dirty read	Non-repeatable	Phantom
Read Uncommitted	✓	✓	✓
Read Committed	—	✓	✓
Repeatable Read	—	—	✓
Serializable	—	—	—

У PostgreSQL за замовчуванням **Read Committed**. Read Uncommitted там працює як Read Committed (брудного читання в Postgres нема взагалі). Repeatable Read у Postgres також захищає від фантомів. Чим вищий рівень, тим більше конфліктів і повторів транзакцій, тож за замовчуванням беруть не найвищий.

Блокування і deadlock. Щоб не псувати чужі зміни, база блокує рядки при UPDATE. Deadlock: дві транзакції чекають одна одну.

```
  t=0  A: update ... where id = 1   (тримає рядок 1)
  t=1  B: update ... where id = 2   (тримає рядок 2)
  t=2  A: update ... where id = 2   (чекає B)
  t=3  B: update ... where id = 1   (чекає A)  -> deadlock
```

Postgres сам виявляє це і скасовує одну з транзакцій помилкою. Захист: завжди брати блокування в одному порядку (наприклад, по зростанню id).

MVCC (у Postgres): кожна транзакція бачить «знімок» даних на свій момент, тому читачі не блокують письменників і навпаки.

> **Практика в репо (sql-drills)**
> У репо вправ на транзакції нема, питання на співбесіді тут майже завжди усні: що таке ACID, які аномалії, що таке deadlock.

Частина 2. NoSQL

8. Які бувають і для чого

NoSQL це не одна технологія, а кілька різних моделей даних, кожна під свій тип задач. Загальне в них: гнучка схема або її відсутність, і зазвичай простіше масштабування по кількох серверах ціною слабших гарантій, ніж у SQL.

Тип	Модель даних	Приклади	Для чого добре	Для чого погано
Key-value	ключ і значення, зазвичай у пам'яті	Redis	кеш, сесії, лічильники, rate limit	складні запити, зв'язки між даними
Документна	JSON-подібні документи	MongoDB	дані зі змінною структурою, каталог, профілі	багато зв'язків і join між документами
Wide-column	рядки з великою кількістю колонок, ключ по партиції	Cassandra	дуже великий потік записів, логи, події	довільні запити, join, транзакції
Граф	вузли і зв'язки між ними	Neo4j	соцмережі, рекомендації, «друзі друзів»	прості табличні дані
Пошукова	інвертований індекс по словах	Elasticsearch	повнотекстовий пошук, аналіз логів	основне сховище даних, транзакції
Time-series	значення з часовою міткою	InfluxDB, TimescaleDB	метрики, IoT, моніторинг	загальні бізнес-дані

Приклад сценарію для кожного: Redis тримає сесію користувача і не дає слати більше 10 запитів на хвилину; MongoDB зберігає товари, у яких різні поля залежно від категорії; Cassandra приймає мільйони подій кліків; Neo4j відповідає «кого ти можеш знати через спільних друзів»; Elasticsearch шукає по тексту статей; Timescale зберігає температуру датчиків щосекунди.

> **Практика в репо (sql-drills)**
> Практики в репо нема: питають «які ти знаєш NoSQL бази і коли яку брати». Відповідай по таблиці вище: тип, приклад, сценарій.

9. SQL чи NoSQL: як вибирати

	SQL (Postgres, MySQL)	NoSQL
Схема	жорстка, задається наперед	гнучка, змінюється разом з даними
Зв'язки і join	основна сила	слабке місце або відсутні
Транзакції	повні ACID	залежить від бази, часто обмежені
Масштабування	в основному вгору (потужніший сервер), горизонтально складніше	горизонтально, по багатьох серверах, закладено в дизайн
Консистентність	сильна	часто eventual (кінцева), налаштовується

CAP-теорема: коли в розподіленій системі стається розрив мережі, доводиться вибирати між узгодженістю даних (Consistency) і доступністю (Availability). Обидва одночасно в момент розриву мати не вийде. Як саме веде себе база, залежить від її налаштувань.

BASE як протилежність ACID: Basically Available (система працює), Soft state (стан може змінюватись без нових запитів), Eventually consistent (репліки зрештою збігаються, але не миттєво).

Правило для відповіді: дані мають зв'язки і важлива коректність (гроші, замовлення) бери SQL. Потрібен швидкий кеш, величезний потік подій, пошук по тексту чи гнучка структура бери відповідну NoSQL. На практиці їх поєднують: Postgres як основне сховище, Redis як кеш, Elasticsearch для пошуку (так звана polyglot persistence).

Коротко: питання, які ставлять усно

Питання	Одним реченням
WHERE чи HAVING?	WHERE до групування по рядках, HAVING після по групах.
INNER чи LEFT JOIN?	INNER лишає тільки пари, LEFT лишає всі рядки лівої і NULL там, де пари нема.
UNION чи UNION ALL?	UNION прибирає дублікати і повільніший, UNION ALL лишає все.
Навіщо індекс і що він коштує?	Швидкий пошук без читання всієї таблиці; сповільнює запис і займає місце.
Чому індекс не спрацював?	Функція над колонкою, LIKE '%x', складений індекс без першої колонки, мала таблиця.
Що таке ACID?	Atomicity, Consistency, Isolation, Durability: усе або нічого, правила збережені, ізоляція, не пропадає після commit.
Що таке deadlock?	Дві транзакції чекають блокування одна одної; база скасовує одну.
Коли NoSQL замість SQL?	Кеш, великий потік подій, пошук, гнучка структура; SQL, де є зв'язки і гроші.